Text-to-Wikidata Entity Linking: a GraphRAG-inspired Pipeline

Goal: a lightweight Python pipeline that identifies meaningful terms (not only named entities) in unstructured text and links each to the right Wikidata concept.

Hypothesis: graph context from Wikidata improves the entity-linking (disambiguation) step. The pipeline runs fully locally (spaCy + small Ollama models), and every design decision below was made based on the experiments done on the candidate generator and the ranker; the resulting architecture is shown in Fig. 1. *For more details on the architecture, implementation and experiments, see `ARCHITECTURE.md` in the GitHub repository. The appendix and references are outside the 2-page limit.*

Dataset: built from ELEVANT [1] (EMNLP 2022, Uni Freiburg), chosen because it annotates entities beyond named entities. Gold QIDs were re-validated against current Wikidata, and the set was stratified into ambiguous vs non-ambiguous mentions, crossed with the NER outcome of step 1 (ambiguous/non-ambiguous $\times$ ner/no-ner). Documents were sampled by taking those with the most mentions, keeping all mentions of a chosen document and breaking ties to keep the ambiguity split closest to even.

1. Text processing and term extraction

- spaCy (small model) splits paragraphs into sentences and extracts POS, noun phrases, lemmas and named entities. Each mention is expanded with its lemma, stem and WordNet synonyms for retrieval.
- **Decision 1 — spaCy alone misses non-named-entity mentions.** spaCy's NER models (small, medium or large) are limited to named entities, while the dataset's entity types — identified first — go beyond them, so the Ollama-served d1-omni:600m was used: a 600 M-parameter encoder (decision model) [2] that answers typed NER questions with a calibrated probability per option in one forward pass. Merging spaCy with the decision model raised mention coverage:

Term extraction	Mention coverage
spaCy (small) alone	61/94 $\approx$ 65%
spaCy + d1-omni:600m	**90/100 = 90%**

2. Candidate retrieval (Wikidata API)

- Each mention form (surface, lemma, stem, synonyms) is queried in parallel; candidates are cached in a SQLite store with QID, label, description, aliases and 1-hop triples. For graph context, each candidate's 1-hop triples are fetched from Wikidata already at this stage. The stage is tuned for recall; selection is left to the ranker.
- **Decision 2 — wbsearchentities over CirrusSearch.** Cirrus full-text search surfaces less prominent entities that merely contain the query words; wbsearchentities ranks by prominence, which matches how mentions are used in text. The mention-only retrieval was selected as the full-text search yielded a lower recall:

Candidate retrieval	Overall recall	Ambiguous	Non-ambiguous
wbsearchentities	**68/90 = 75.6%**	27/44 = 61.4%	41/46 = 89.1%
CirrusSearch (full-text)	35/90 = 38.9%	19/44 = 43.2%	16/46 = 34.8%

- **Insight.** Candidate recall is the ceiling on end-to-end accuracy, and the misses are more on ambiguous mentions (61.4% vs 89.1%) — on those, the generator, not the ranker, is the bottleneck. Recall-oriented query expansion (lemma, stem, synonyms) plus prominence-ranked search hold overall recall at 75.6%.

3. Knowledge graph enrichment

- For every candidate, its Wikidata 1-hop triples are fetched and stored. Per document, a NetworkX graph connects all candidates of all mentions: nodes are QIDs, edges are Wikidata properties, both keeping their RDF terms (wd:/wdt: IRIs), following the RDF/SKOS framing.
- This graph is the disambiguation signal: correct candidates pull together into a connected cluster — e.g. Belarus, Germany and Ireland link via "diplomatic relation" — while wrong candidates (the element germanium, the person Samuel Ireland) stay isolated (Fig. 2).

4. GraphRAG-based candidate selection — ranker architecture and experiments

GraphRAG framing: a RAG pipeline usually retrieves from a dedicated vector or graph database; here Wikidata itself is the graph database, and retrieval from it happens at the candidate-generation phase (steps 2–3).

Base scorer (all variants): cosine similarity between the embedding of "mention + sentence" and each candidate rendered as "label: description (aliases)"; the embeddings are created with the embeddinggemma model (`embeddinggemma:latest`, served via Ollama). Added to this are small lexical priors: Wikidata's own search rank (0.15/rank), exact label match (+0.10) and a Levenshtein penalty (-0.05). GraphRAG enters in two ways:

- **Retrieval-augmentation:** each candidate's graph neighbourhood is verbalised into up to 12 facts and appended to the candidate text before embedding.
- **Graph coherence bonus (weight 0.2):** a candidate directly connected to another mention's candidate scores 1.0, a shared neighbour 0.5, an isolated candidate 0.

Seven ranker variants (cr_v1–v7) were built on this scorer and compared on the full stratified run:

Ranker	Context side	Candidate side	Scorer	Overall acc/MRR	Ambiguous	Non-amb.	Time
v1 embed	mention + sentence	label: description (aliases)	embeddinggemma cosine + lexical priors	57.8% / 0.64	40.9% / 0.48	73.9% / 0.79	14 s
v2 embed + graph	mention + sentence	v1 + verbalised graph facts	v1 + coherence bonus (0.2)	**63.3%** / **0.67**	**47.7%** / **0.52**	**78.3%** / **0.82**	19 s
v3 d1	mention + sentence	candidates as options	d1 choice question, calibrated probability	62.2% / 0.67	45.5% / 0.52	**78.3%** / **0.82**	15 s
v4 d1 + graph	mention + sentence	options + graph facts	v3 + coherence bonus	60.0% / 0.66	45.5% / 0.52	73.9% / 0.79	14 s
v5 AMR graph	AMR graph of the sentence, verbalised (amrlib)	label, description, aliases + triples	embeddinggemma cosine, graph-to-graph	60.0% / 0.64	47.7% / 0.51	71.7% / 0.76	858 s
v6 AMR + d1	AMR facts of the sentence	options + graph facts	d1 probability + coherence bonus	56.7% / 0.63	38.6% / 0.47	73.9% / 0.79	28 s
v7 graph-embed	centroid of other mentions' candidate vectors	NetMF-style structural node embedding	cosine to centroid	57.8% / 0.64	38.6% / 0.47	76.1% / 0.80	19 s

Decisions made from these experiments:

- **Embedding vs decision-model ranking.** v3/v4 phrase ranking as one d1 choice question (table above). The d1 rankers land just behind the embeddings (62.2% / 60.0% vs 63.3%), but graph context only helps the embedding family (v1→v2, +5.5 points overall); for d1 it slightly hurts (v3→v4, −2.2) — the verbalised facts lengthen the options without adding discriminative evidence for the decision model.
- **Verbalised vs structural graph embeddings.** v7 swaps the verbalised facts for structural node embeddings; it matched the no-graph baseline overall (57.8%) but lost to verbalised graph context on ambiguous mentions (38.6% vs 47.7%).
- **Negative results — AMR (Abstract Meaning Representation [3]) graphs.** v5 (graph-to-graph matching) ties v2 on ambiguous mentions (47.7%) but trails overall (60.0% vs 63.3%) and takes 858 s — $\sim$45$\times$ slower — ruling it out. v6 (the same AMR facts in a d1 choice question) drops to 56.7%, joint-worst on ambiguous mentions (38.6%, with v7): stacking verbalised graphs on both sides dilutes the evidence instead of sharpening it.
- **Final ranker: v2 (embeddings + verbalised graph facts + coherence bonus).** Graph context lifts overall accuracy 57.8% $\rightarrow$ 63.3% and ambiguous accuracy 40.9% $\rightarrow$ 47.7% over the no-graph baseline. The top candidate is returned only when evidence is sufficient, otherwise alternatives are surfaced.
- **Insight.** Graph context gives its gain exactly where disambiguation is hardest — ambiguous mentions (+6.8 points, v1→v2) — but only when paired with the embedding ranker. How the graph is used matters more than the ranking model: verbalised facts beat both structural embeddings (v7) and AMR graphs (v5/v6). MRR above accuracy (0.67 vs 63.3%) means the gold entity is usually just below a wrong rank 1, so the surfaced alternatives often contain it.

5. Output and evaluation

- Output is structured JSON per mention: original term and sentence, selected concept with QID, rank, score, confidence and supporting evidence, plus the ranked alternative candidates.
- Evaluation compares the no-graph baseline against the graph-context ranker on the stratified set (table above): the gain is largest where disambiguation is hardest, confirming the hypothesis.

6. Future work

- The ranker can be experimented with different embedding models and other decoder-based models.
- Different graph embeddings can be applied in place of NetMF — e.g. translational models such as TransE over the candidate graphs.
- For GraphRAG, a local graph database (e.g. FalkorDB) or a vector database can hold 1–4-hop triples fetched from Wikidata, queried at the ranker phase — more precise graph information than the current 1-hop store.
- More work on the AMR front (v5): the sentence itself is converted into a graph, and graph-to-graph matching happens at the ranking phase — promising on ambiguous mentions, but it needs a faster parser.

Code and experiments: `https://github.com/itisha2/text-to-wikidata-linking` (main branch) — `src/` holds the text-preprocessing, candidate-generator and ranker code; `src/ranker/` has all seven ranker experiments (`cr_v1`–`cr_v7`). `inference_pipeline.ipynb` implements the final entity-linking pipeline by importing these modules and can be run against any text or paragraph; candidate retrieval may take a while, as its latency depends on the Wikidata search API.

AI usage declaration. AI assistance was used for code polishing and for paraphrasing the write-up. The dataset selection, the candidate-generator and ranker experiment design, the decision and evaluation criteria, and the baseline code implementation are the author's own.

Appendix

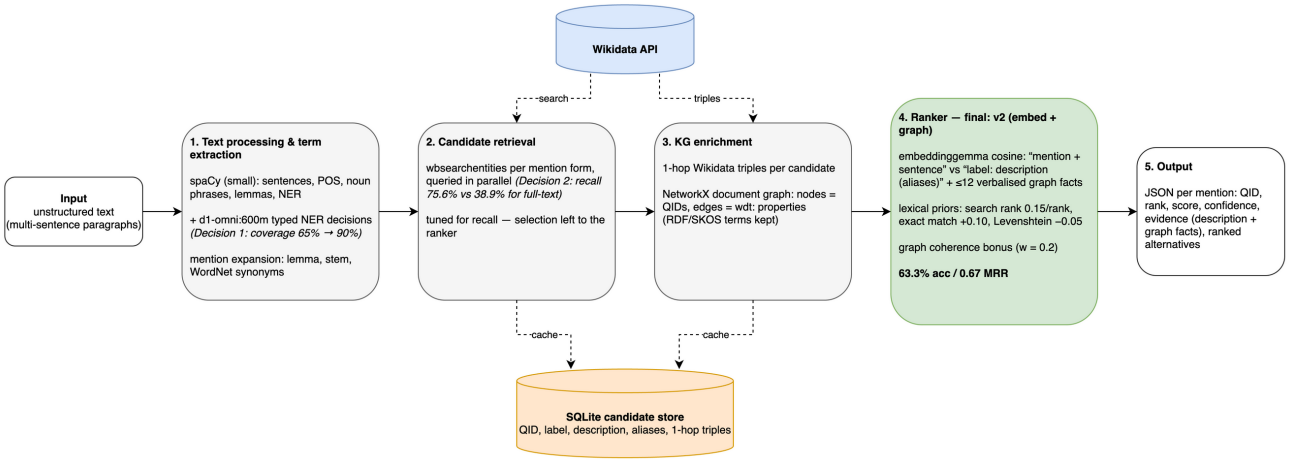


Figure 1: Final pipeline architecture after Decisions 1–2 and the ranker experiments (final ranker: v2).

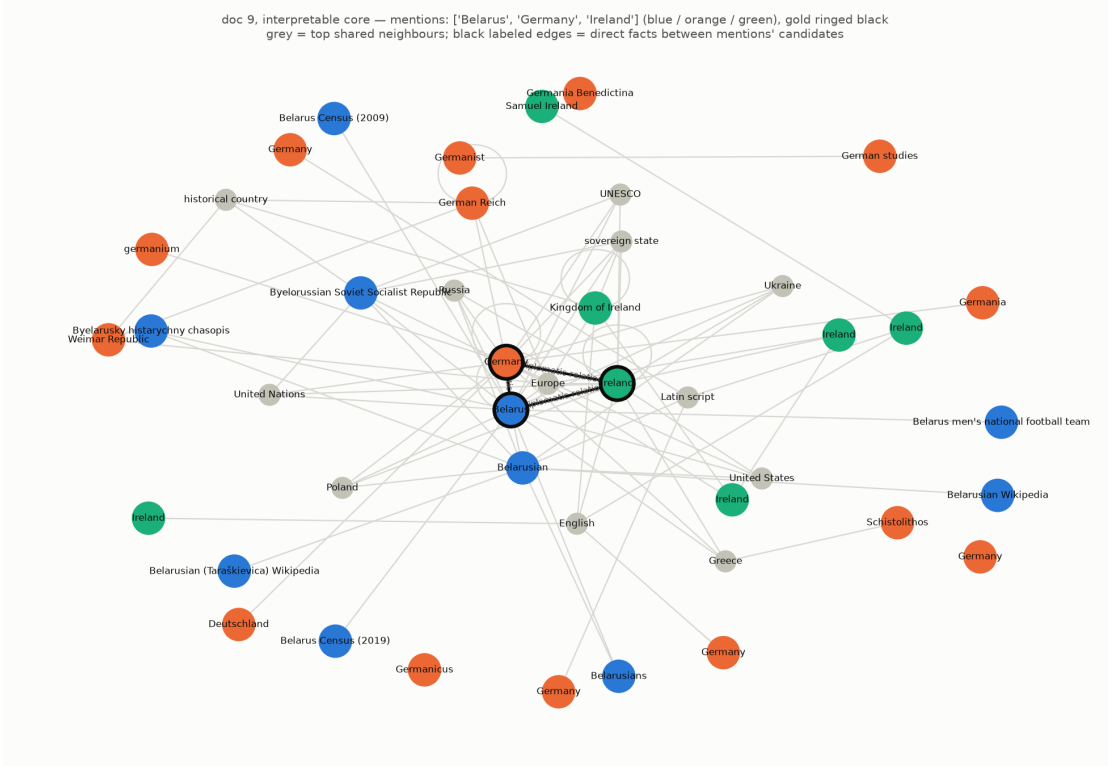


Figure 2: Document-level candidate graph (document 9). The correct candidates (Belarus, Germany, Ireland) form a tightly connected core via "diplomatic relation" edges, while wrong candidates with the same surface forms (the chemical element germanium, the person Samuel Ireland) have no connections to the rest of the document and drift to the periphery. The coherence bonus scores exactly this structure.

References

[1] Hannah Bast, Matthias Hertel, Natalie Prange. 2022. ELEVANT: A Fully Automatic Fine-Grained Entity Linking Evaluation and Analysis Tool. In *Proceedings of EMNLP 2022: System Demonstrations*. `https://aclanthology.org/2022.emnlp-demos.8/`

[2] d1-omni:600m decision model. Ollama model library. `https://ollama.com/library/d1-omni:600m`

[3] Peitao Han, Lis Pereira, Fei Cheng, Wan Jou She, Eiji Aramaki. 2025. AMR-RE: Abstract Meaning Representations for Retrieval-Based In-Context Learning in Relation Extraction. In *Proceedings of NAACL 2025, Volume 4: Student Research Workshop*. `https://aclanthology.org/2025.naacl-srw.33/`